



El libro del curso

Python con Pokémon

Aprendé a programar desde cero, paso a paso, con Linux, Python y mucha aventura.

Y convertite en Campeón de Kanto

Manual + libro de Python · `generar_manual.py`

Índice

0. Cómo usar este libro y el curso

- 0.1. ¿Qué es este proyecto?
- 0.2. Instalación
- 0.3. La Liga Pokémon: jugá tu progreso
- 0.4. El método: cómo trabajar cada semana

1. Linux: Fundamentos

- 1.1. ¿Qué es Linux?
- 1.2. La terminal: tu Pokédex del sistema
- 1.3. El sistema de archivos: el mapa del mundo
- 1.4. Los comandos básicos (tus primeros ataques)
- 1.5. Rutas absolutas vs relativas
- 1.6. Permisos básicos
- 1.7. Resumen de la semana
- 1.8. ¿Y ahora qué?

2. Linux: Intermedio

- 2.1. Analogía: tu base de operaciones
- 2.2. El editor nano
- 2.3. Variables de entorno
- 2.4. Scripts bash básicos
- 2.5. Redirección: > y >>
- 2.6. Pipes: |
- 2.7. grep: buscar texto
- 2.8. find: encontrar archivos
- 2.9. Procesos: ps y kill
- 2.10. chmod: cambiar permisos
- 2.11. Usuarios y permisos: sudo
- 2.12. apt: instalar programas
- 2.13. SSH: conectarte a otra máquina
- 2.14. Resumen de la semana
- 2.15. ¿Y ahora qué?

3. Python: Introducción

- 3.1. ¿Qué es Python?

- 3.2. ¿Cómo se corre Python?
- 3.3. `print()`: mostrar cosas en pantalla
- 3.4. Variables: cajas con nombre
- 3.5. Tipos de datos
- 3.6. `input()`: pedirle datos al usuario
- 3.7. Conversión de tipos
- 3.8. Comentarios
- 3.9. f-strings: la forma copada de armar texto
- 3.10. Resumen de la semana
- 3.11. ¿Y ahora qué?

4. Python: Control de Flujo

- 4.1. Analogía: decisiones en batalla
- 4.2. `if`: tomar decisiones
- 4.3. Operadores de comparación
- 4.4. `elif` y `else`: más caminos
- 4.5. Operadores lógicos: `and`, `or`, `not`
- 4.6. `while`: repetir mientras...
- 4.7. `for`: repetir una cantidad de veces
- 4.8. `break` y `continue`
- 4.9. Resumen de la semana
- 4.10. ¿Y ahora qué?

5. Python: Funciones

- 5.1. Analogía: los ataques aprendidos
- 5.2. Definir una función con `def`
- 5.3. Parámetros: darle datos a la función
- 5.4. `return`: devolver un resultado
- 5.5. Valores por defecto
- 5.6. Scope: dónde vive cada variable
- 5.7. Funciones `lambda`: funciones express
- 5.8. Docstrings: documentar tu función
- 5.9. Recursión: una función que se llama a sí misma
- 5.10. Resumen de la semana
- 5.11. ¿Y ahora qué?

6. Python: Listas y Colecciones

- 6.1. Analogía: tu equipo Pokémon
- 6.2. Listas: colección ordenada y modificable
- 6.3. Métodos de listas (las acciones de tu equipo)
- 6.4. Tuplas: colección que NO se modifica
- 6.5. Sets: colección SIN duplicados
- 6.6. Diccionarios: pares clave → valor
- 6.7. Comprensiones de listas: crear listas en una línea
- 6.8. enumerate y zip
- 6.9. Resumen de la semana
- 6.10. ¿Y ahora qué?

7. Python: Cadenas y Archivos

- 7.1. Analogía: la Pokédex que recuerda
- 7.2. Métodos de strings (cadenas de texto)
- 7.3. Slicing: rebanar strings
- 7.4. Abrir archivos con open()
- 7.5. with: la forma correcta de abrir archivos
- 7.6. CSV: datos en filas y columnas
- 7.7. Manejo de excepciones: try / except
- 7.8. Resumen de la semana
- 7.9. ¿Y ahora qué?

8. Git: Control de Versiones

- 8.1. Analogía: Git es "guardar la partida"
- 8.2. ¿Por qué usar Git?
- 8.3. Configuración inicial (una sola vez)
- 8.4. Crear un repositorio: git init
- 8.5. Ver el estado: git status
- 8.6. Los 3 pasos de un guardado
- 8.7. Ver la historia: git log
- 8.8. Ramas (branches): líneas temporales alternativas
- 8.9. Fusionar ramas: git merge
- 8.10. GitHub: la PC de Bill en la nube
- 8.11. .gitignore: lo que Git debe ignorar
- 8.12. El flujo de trabajo típico (memorízalo)

8.13. Resumen de la semana

8.14. ¿Y ahora qué?

9. Python: POO Introducción

9.1. Analogía: el molde y los Pokémon

9.2. Definir una clase

9.3. init: el constructor

9.4. Atributos: los datos del objeto

9.5. Métodos: las acciones del objeto

9.6. str: cómo se muestra el objeto

9.7. repr: la versión para programadores

9.8. Encapsulamiento básico

9.9. Resumen de la semana

9.10. ¿Y ahora qué?

10. Python: POO Avanzado

10.1. Analogía: las especies y los tipos

10.2. Herencia: heredar de una clase padre

10.3. super(): llamar al padre

10.4. Polimorfismo: cada uno responde a su manera

10.5. Métodos de clase y estáticos

10.6. Propiedades con @property

10.7. Clases abstractas con abc

10.8. Resumen de la semana

10.9. ¿Y ahora qué?

11. Python: Módulos y pip

11.1. Analogía: las MT (Máquinas Técnicas)

11.2. import: traer un módulo

11.3. Módulos estándar útiles

11.4. Crear tu propio módulo

11.5. pip: instalar librerías externas

11.6. venv: entornos virtuales

11.7. requirements.txt

11.8. requests: consumir una API

11.9. Resumen de la semana

11.10. ¿Y ahora qué?

12. Mapa de temas del curso

- 12.1. Fase 1 — Linux
- 12.2. Fase 2 — Python básico
- 12.3. Descanso — Git
- 12.4. Fase 3 — Python avanzado
- 12.5. Fase 4 — Proyectos

13. Ayuda: tests, preguntas frecuentes y glosario

- 13.1. Cómo correr los tests
- 13.2. Preguntas frecuentes
- 13.3. Glosario

CAPÍTULO 0. Cómo usar este libro y el curso

¡Bienvenido! Este libro tiene **dos caras**. Por un lado es el **manual** del proyecto: te explica cómo instalarlo y cómo avanzar. Por otro lado es un **libro completo** que te enseña Linux y Python desde cero: contiene **toda la teoría del curso**, con explicaciones paso a paso, analogías Pokémon y muchos ejemplos.

Está pensado para alguien que **nunca programó**. No hace falta saber nada de antes.

0.1. ¿Qué es este proyecto?

Es un curso completo de **Linux** y **Python**, dividido en 12 semanas (más una semana de descanso de Git) y 4 proyectos finales. Cada concepto se explica con una analogía Pokémon, cada ejercicio usa Pokémon, y cada proyecto es algo que un Entrenador usaría de verdad.

Los capítulos de teoría de este libro son exactamente las lecciones del curso: podés leer el libro de principio a fin, o usar cada semana del curso por separado.

0.2. Instalación

Abrí una terminal dentro de la carpeta del proyecto y corré:

```
BASH
bash setup.sh
```

Eso crea un entorno virtual (una cajita aislada para las librerías del curso) e instala lo necesario. Después, cada vez que vayas a trabajar, lo activás:

```
BASH
source venv/bin/activate
```

CUIDADO

Si `setup.sh` falla, ábrilo con un editor: está comentado línea por línea. Las dos primeras semanas son de Linux y no necesitan Python para los ejercicios de terminal.

0.3. La Liga Pokémon: jugá tu progreso

La mejor forma de hacer el curso. En vez de "hacer la tarea", subís de nivel. Hay un único programa que es tu centro de mando:



```
BASH
python aventura.py
```

Desde ese único archivo podés:

- **Elegir un capítulo para jugar:** lanza el juego interactivo de esa semana. Si tenés progreso, **continuás donde lo dejaste**; si ya lo completaste, te ofrece **reintentarlo**.
- **Entrenar** una semana: corre tus ejercicios y te da EXP por cada test que pasás. Si algo falla, te dice **qué ejercicio** y una pista.
- **Combates de gimnasio:** con una medalla en mano, retás al líder con un desafío integrador más difícil.
- Ver tu **Tarjeta de Entrenador**, las **8 medallas**, los **logros** y el mapa.
- Mantener tu **racha** diaria.

Cuando consigas las 8 medallas (y venzas a los 8 líderes), ¡sos **Campeón de Kanto!**

0.4. El método: cómo trabajar cada semana

1. **Leé** la teoría del tema (este libro la tiene completa).
2. **Resolvé** los ejercicios (escribí tu código donde dice `# TU CÓDIGO ACÁ`).
3. **Probá** con los tests (ver el [capítulo de ayuda](#)).
4. **Jugá** el `interactivo.py` de la semana.
5. **Entrená** esa semana en la Liga para ganar EXP.

TIP

Regla de oro: equivocarse es parte del juego. Escribí el código vos mismo, no copies y pegues: tipear te enseña.

CAPÍTULO 1. Linux: Fundamentos

META

Meta de la semana: moverte por tu computadora usando solo texto, sin mouse. Vas a aprender los comandos básicos de la terminal de Linux.

1.1. ¿Qué es Linux?

Linux es un sistema operativo, igual que Windows o macOS. La diferencia es que es **libre y gratuito**, y lo usan casi todos los servidores del mundo, celulares Android, supercomputadoras... y casi todos los programadores.

Si estás en Windows, podés usar **WSL** (Windows Subsystem for Linux) para tener Linux adentro de Windows. Si estás en Mac, la terminal es muy parecida.

Analogía Pokémon

Pensá en Linux como el **mundo Pokémon** completo: las rutas, las ciudades, los gimnasios. Vos sos el Entrenador que lo recorre.

Y la **terminal** es tu **Pokédex**: una herramienta de texto donde escribís comandos y el sistema te responde. Al principio parece complicada, pero es la herramienta más poderosa que vas a tener.

1.2. La terminal: tu Pokédex del sistema

La terminal (o "consola") es una ventana donde escribís **comandos** y presionás Enter. El sistema ejecuta lo que pediste y te muestra el resultado.

Cuando la abrís, ves algo así:



Eso se llama el **prompt**. Te dice:

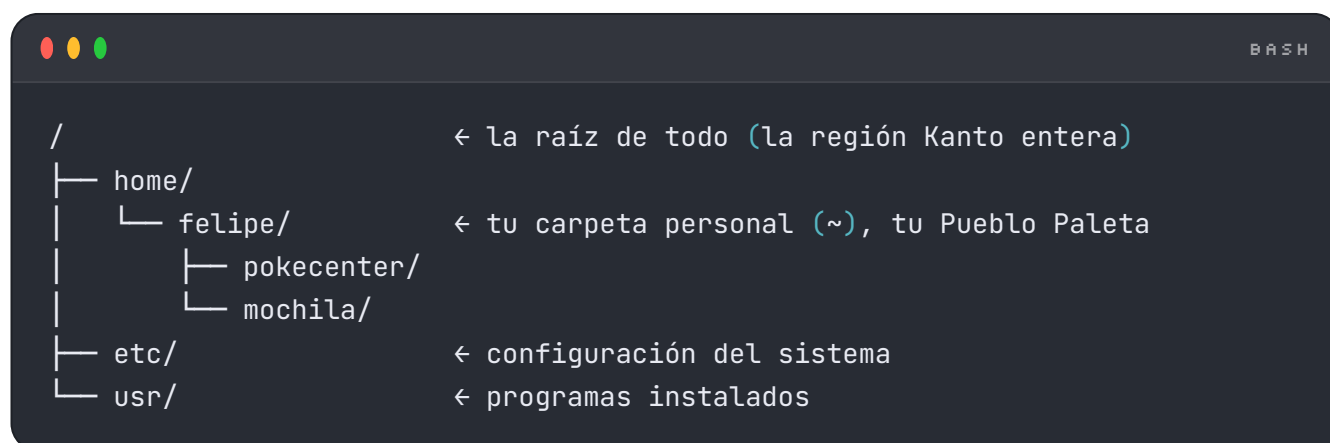
- **felipe** → tu usuario (el Entrenador).
- **maquina** → el nombre de la computadora.
- **~** → en qué carpeta estás (**~** significa "tu carpeta personal", tu home).
- **\$** → "estoy listo, escribí tu comando".

Cada comando es un "ataque"

Así como Pikachu tiene **Impactrueno** o **Ataque Rápido**, vos tenés comandos. Cada uno hace una cosa específica. Aprenderlos es como aprender los movimientos de tu Pokémon: al principio pocos, después un montón.

1.3. El sistema de archivos: el mapa del mundo

Todo en Linux está organizado en **carpetas** (también llamadas "directorios"), una adentro de otra, como cajas dentro de cajas. Esto forma un **árbol**.



- `/` es la **raíz**: el punto de partida de todo.
- `~` es tu **home**: tu casa, donde guardás tus cosas.

1.4. Los comandos básicos (tus primeros ataques)

`pwd` — ¿Dónde estoy?

`pwd` significa print working directory. Te dice en qué carpeta estás parado ahora mismo. Es tu **mapa del pueblo actual**.

```

B A S H

pwd
  
```

Salida de ejemplo:

```

B A S H

/home/felipe
  
```

ls — Mirar alrededor

ls (list) muestra qué hay en la carpeta actual: archivos y otras carpetas. Es como abrir los ojos y ver qué Pokémon hay en el pasto.

```
ls
```

Con **flags** (opciones) ves más detalle. Un flag es como un objeto que potencia el ataque:

```
ls -l      # formato largo: permisos, tamaño, fecha
ls -a      # muestra TODO, incluso archivos ocultos (empiezan con .)
ls -la     # combinás los dos
```

cd — Viajar a otra carpeta

cd (change directory) te mueve de una carpeta a otra. Es **viajar por una ruta** del mapa.

```
cd pokecenter  # entrás a la carpeta pokecenter
cd ..          # subís un nivel (volvés a la carpeta de arriba)
cd ~           # volvés a tu home (tu Pueblo Paleta)
cd /           # vas a la raíz de todo
cd -           # volvés a la carpeta donde estabas antes
```

mkdir — Crear una carpeta

mkdir (make directory) crea una carpeta nueva. Es como **construir un edificio nuevo** en el pueblo.

```
mkdir pokecenter  # crea la carpeta pokecenter
mkdir gimnasio pokemart  # crea dos carpetas de una
mkdir -p ruta/larga/nueva  # crea carpetas anidadas de un saque (-p)
```

`touch` y `echo` — Crear archivos

`touch` crea un archivo vacío. `echo` imprime texto, y con `>` lo podés guardar en un archivo.

```

touch pikachu.txt           # crea un archivo vacío
echo "Pikachu, tipo Eléctrico" # imprime el texto en pantalla
echo "Pikachu" > pikachu.txt # escribe el texto DENTRO del archivo

```

`cat` — Leer un archivo

`cat` muestra el contenido de un archivo en pantalla. Es como **leer la entrada de la Pokédex** de un Pokémon.

```

cat pikachu.txt

```

`cp` — Copiar

`cp` (copy) hace una copia de un archivo o carpeta. Como usar una **Pokéball para duplicar** (bueno, casi).

```

cp pikachu.txt pikachu-copia.txt # copia un archivo
cp -r pokecenter pokecenter-backup # copia una carpeta entera (-r =
recursivo)

```

`mv` — Mover o renombrar

`mv` (move) mueve un archivo de lugar, o lo renombra si le das un nombre nuevo.

```

mv pikachu.txt mochila/ # mueve el archivo a la carpeta mochila
mv pikachu.txt raichu.txt # lo renombra (Pikachu evolucionó a Raichu )

```

rm — Borrar (¡CUIDADO!)

rm (remove) borra archivos. **En Linux NO hay papelera de reciclaje**: lo que borras, se va para siempre. Es como liberar un Pokémon: no vuelve.

```
BASH
rm pikachu.txt          # borra un archivo
rm -r pokecenter        # borra una carpeta y todo su contenido (-r)
```

CUIDADO

Nunca, jamás corras `rm -rf /`. Eso intenta borrar TODO el sistema. Es el "Autodestrucción" definitivo. No lo hagas nunca.

1.5. Rutas absolutas vs relativas

Una **ruta** es la dirección de un archivo o carpeta. Hay dos formas de escribirla:

Ruta absoluta — la dirección completa

Empieza desde la raíz `/`. Funciona desde cualquier lugar, como dar tu dirección completa con código postal.

```
BASH
cd /home/felipe/pokecenter
```

Ruta relativa — desde donde estás parado

No empieza con `/`. Es relativa a tu posición actual, como decir "dobla a la izquierda en la esquina".

```
BASH
cd pokecenter    # entra a pokecenter que está acá
cd ../mochila     # sube uno y entra a mochila
```

Símbolos clave:

- `.` → la carpeta actual (acá mismo).
- `..` → la carpeta de arriba (el nivel anterior).

- `~` → tu home.
- `/` → la raíz.

1.6. Permisos básicos

En Linux, cada archivo tiene **permisos** que dicen quién puede leerlo, escribirlo o ejecutarlo. Es como decidir quién puede entrar a tu gimnasio.

Cuando corrés `ls -l`, ves algo así:

```

-rwxr-xr--  1 felipe felipe  220 jun  4 10:00 pikachu.txt
  
```

Esa primera columna `-rwxr-xr--` son los permisos:

- Primer carácter: `-` archivo, `d` directorio.
- Después, en grupos de 3: **dueño**, **grupo**, **otros**.
- `r` = leer (read), `w` = escribir (write), `x` = ejecutar (execute).

Por ahora solo necesitas **reconocerlos**. En la semana 2 vas a aprender a cambiarlos con `chmod`.

1.7. Resumen de la semana

Comando	Qué hace	Analogía Pokémon
<code>pwd</code>	Dice dónde estás	Mirar el mapa
<code>ls</code>	Lista archivos y carpetas	Ver qué hay alrededor
<code>cd</code>	Cambia de carpeta	Viajar por una ruta
<code>mkdir</code>	Crea una carpeta	Construir un edificio
<code>touch</code>	Crea un archivo vacío	Conseguir un objeto nuevo
<code>echo</code>	Imprime texto	Hablar
<code>cat</code>	Muestra un archivo	Leer la Pokédex
<code>cp</code>	Copia	Duplicar

Comando	Qué hace	Analogía Pokémon
<code>mv</code>	Mueve o renombra	Mudarse / evolucionar
<code>rm</code>	Borra (¡sin papelera!)	Liberar (no vuelve)

Rutas: absoluta empieza con `/`, relativa desde donde estás. `.` = acá, `..` = arriba, `~` = home.

Permisos: `r` leer, `w` escribir, `x` ejecutar.

1.8. ¿Y ahora qué?

1. Abrí `ejercicios.md` y hacé los 15 desafíos en tu terminal de verdad.
2. Si te trabás, mirá `soluciones.md`.
3. Jugá el `interactivo.py` para practicar sin miedo a romper nada:

```
python interactivo.py
```

ÁNIMO

"Todo gran Entrenador empezó sin saber usar la Pokédex. Vos ya diste el primer paso."

CAPÍTULO 2. Linux: Intermedio

META

Meta de la semana: dejar de ser un Entrenador novato y convertirte en uno que **configura su propia base de operaciones**. Vas a editar archivos, escribir tus primeros scripts, encadenar comandos y manejar procesos.

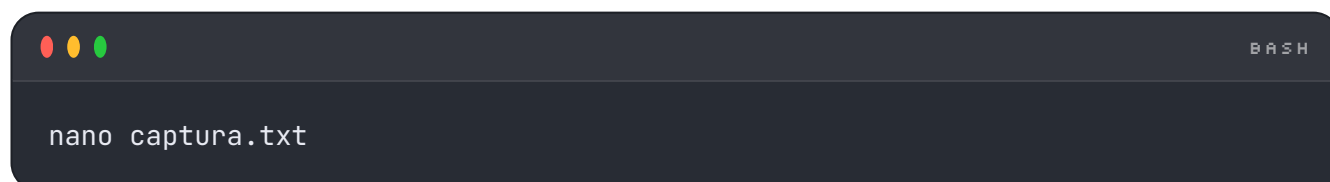
2.1. Analogía: tu base de operaciones

En la semana 1 aprendiste a caminar por el mundo Pokémon. Esta semana **construís tu base**: automatizás tareas, organizás tu información y le das poder a tu Pokédex.

Un Entrenador avanzado no captura Pokémon uno por uno a mano: arma **máquinas** (scripts) que hacen el trabajo repetitivo por él. Eso es lo que vas a aprender.

2.2. El editor nano

Hasta ahora creabas archivos con `touch` y `echo`. Para escribir texto largo, usás un **editor**. El más fácil es **nano**.



```
nano captura.txt
```

Esto abre el editor. Escribís lo que quieras, y abajo ves los atajos:

- `^O` (Ctrl+O) → guardar (write Out). Te pide confirmar el nombre, apretás Enter.
- `^X` (Ctrl+X) → salir.
- `^K` → cortar una línea, `^U` → pegar.
- `^W` → buscar texto.

TIP

El `^` significa la tecla **Ctrl**. Así que `^X` = Ctrl + X.

2.3. Variables de entorno

Una **variable de entorno** es un dato que el sistema guarda con un nombre, disponible para todos los programas. Es como tener **objetos clave guardados en tu mochila** que cualquier programa puede consultar.

```

B A S H

echo $HOME      # muestra la ruta de tu home
echo $USER      # tu nombre de usuario
echo $PATH      # las carpetas donde el sistema busca comandos

```

Crear tu propia variable:

```

B A S H

ENTRENADOR="Ash"      # creamos la variable (¡sin espacios alrededor del
=!)
echo $ENTRENADOR      # la usamos con $ adelante
export ENTRENADOR     # la hacemos visible para otros programas

```

2.4. Scripts bash básicos

Un **script** es un archivo de texto con una lista de comandos que se ejecutan en orden. Es tu **máquina automática**.

Creá un archivo `ho!a.sh`:

```

B A S H

#!/usr/bin/env bash
# La primera línea (el "shebang") dice qué programa ejecuta el script.

echo "¡Ho!a, Entrenador!"
echo "Hoy es un buen día para atrapar Pokémon."

```

Para correrlo, primero le das permiso de ejecución y después lo ejecutás:

```

B A S H

```

```
chmod +x hola.sh      # +x = agregar permiso de ejecución
./hola.sh             # ./ significa "ejecutá este archivo de acá"
```

Variables y argumentos en scripts

```
#!/usr/bin/env bash
NOMBRE="Pikachu"
echo "Mi Pokémon es $NOMBRE"

# $1 es el primer argumento que le pasás al script.
echo "Capturaste a: $1"
```

Lo ejecutás así: `./script.sh Charizard` → imprime "Capturaste a: Charizard".

2.5. Redirección: `>` y `>>`

La **redirección** manda la salida de un comando a un archivo en vez de a la pantalla.

```
echo "Pikachu" > equipo.txt      # > CREA o REEMPLAZA el archivo
echo "Charizard" >> equipo.txt    # >> AGREGA al final (no borra lo anterior)
```

CUIDADO

Cuidado: `>` pisa todo lo que había. `>>` suma. Confundirlos borra datos.

2.6. Pipes: `|`

Un **pipe** (tubería) conecta la salida de un comando con la entrada de otro. Es como hacer que **dos Pokémon combinen ataques**.



BASH

```
ls | wc -l
# ls lista archivos, wc -l cuenta cuántas líneas → cuántos archivos hay
cat equipo.txt | sort # muestra el equipo ordenado alfabéticamente
```

2.7. grep: buscar texto

grep busca un texto dentro de archivos o de lo que le llega por pipe. Es tu **detector de Pokémon**.

```
grep "Pikachu" equipo.txt # muestra las líneas que contienen
"Pikachu"
grep -i "pikachu" equipo.txt # -i = ignora mayúsculas/minúsculas
grep -r "Fuego" carpeta/ # -r = busca recursivamente en una carpeta
cat pokedex.txt | grep "Electrico" # buscar combinando con un pipe
```

2.8. find: encontrar archivos

grep busca **dentro** de archivos. **find** busca **los archivos en sí**.

```
find . -name "*.txt" # busca todos los .txt desde la carpeta actual
find ~ -name "pikachu*" # busca en tu home archivos que empiecen con
pikachu
find . -type d # busca solo carpetas (type d = directory)
```

2.9. Procesos: ps y kill

Un **proceso** es un programa corriendo. Como un Pokémon activo en batalla.

```
ps # muestra tus procesos
ps aux # muestra TODOS los procesos del sistema (muchísima info)
ps aux | grep python # filtra para ver solo los de python
```

Si un programa se cuelga, lo "debilitas" con `kill`:

```
BASH
kill 1234          # 1234 es el PID (número de proceso). Pedido amable de
                  # cerrar.
kill -9 1234       # -9 = cierre forzado. El "golpe crítico". Usar solo si
                  # hace falta.
```

2.10. chmod: cambiar permisos

En la semana 1 aprendiste a **leer** los permisos. Ahora los **cambiás** con `chmod`.

```
BASH
chmod +x script.sh # agrega permiso de ejecución (la forma más común)
chmod -x script.sh # quita el permiso de ejecución
chmod 755 script.sh
# forma numérica (avanzada): dueño rwx, grupo y otros r-x
```

La forma numérica: cada dígito es la suma de `r`=4, `w`=2, `x`=1.

- `7` = 4+2+1 = rwx (todo)
- `5` = 4+0+1 = r-x (leer y ejecutar)
- `6` = 4+2+0 = rw- (leer y escribir)

2.11. Usuarios y permisos: sudo

Linux es **multiusuario**. El usuario todopoderoso se llama **root** (el Profesor Oak del sistema: tiene acceso a todo).

`sudo` ejecuta UN comando como root. Te pide tu contraseña.

```
BASH
sudo apt update # actualizar la lista de programas (necesita permisos de
root)
```

CUIDADO

`sudo` es poderoso. Con gran poder viene gran responsabilidad. No corras comandos con `sudo` que no entiendas.

2.12. apt: instalar programas

En Ubuntu/Debian, `apt` es el **PokéMart**: ahí instalás programas.

```

sudo apt update           # actualiza la lista de programas disponibles
sudo apt install cowsay   # instala un programa (en este caso, una vaca
que habla)
sudo apt remove cowsay    # lo desinstala

```

2.13. SSH: conectarte a otra máquina

SSH (Secure Shell) te deja controlar **otra computadora** por la terminal, de forma segura. Es como usar tu Pokédex para conectarte a un **gimnasio remoto**.

```

ssh entrenador@192.168.1.50  # te conectás a esa máquina con ese usuario

```

Te va a pedir contraseña, y después tenés una terminal en la máquina remota. Esto es la base de cómo se administran los servidores del mundo.

TIP

Por ahora solo necesitás saber **qué es**. Cuando tengas un servidor (o una Raspberry Pi), lo vas a usar todo el tiempo.

2.14. Resumen de la semana

Comando

Qué hace

`nano archivo`

Edita un archivo de texto

Comando	Qué hace
<code>\$VARIABLE</code> / <code>export</code>	Variables de entorno
<code>chmod +x</code>	Da permiso de ejecución
<code>./script.sh</code>	Ejecuta un script
<code>></code> / <code>>></code>	Redirige salida (reemplaza / agrega)
<code>\ </code> (pipe)	Conecta salida de un comando con otro
<code>grep</code>	Busca texto dentro de archivos
<code>find</code>	Busca archivos por nombre/tipo
<code>ps</code> / <code>kill</code>	Ver y cerrar procesos
<code>sudo</code>	Ejecuta como root
<code>apt</code>	Instala/desinstala programas
<code>ssh</code>	Te conecta a otra máquina

2.15. ¿Y ahora qué?

1. Hacé los ejercicios de `ejercicios.md` en tu terminal.
2. Jugá el `interactivo.py`: te guía a escribir tu **primer script bash real**.

```
python interactivo.py
```

1. Poné a prueba lo aprendido con el quiz:

```
python quiz.py
```

ÁNIMO

"Un Entrenador que automatiza su trabajo tiene más tiempo para entrenar. Trabaja inteligente, no solo duro."

CAPÍTULO 3. Python: Introducción

META

Meta de la semana: escribir tus primeros programas en Python. Vas a aprender a guardar datos en variables, mostrar cosas en pantalla y pedirle información al usuario.

3.1. ¿Qué es Python?

Python es un lenguaje de programación. Un "lenguaje" es la forma en que le das **órdenes a la computadora**. Python es famoso porque se lee casi como inglés, es fácil de aprender y se usa para TODO: webs, videojuegos, inteligencia artificial, análisis de datos...

Analogía Pokémon

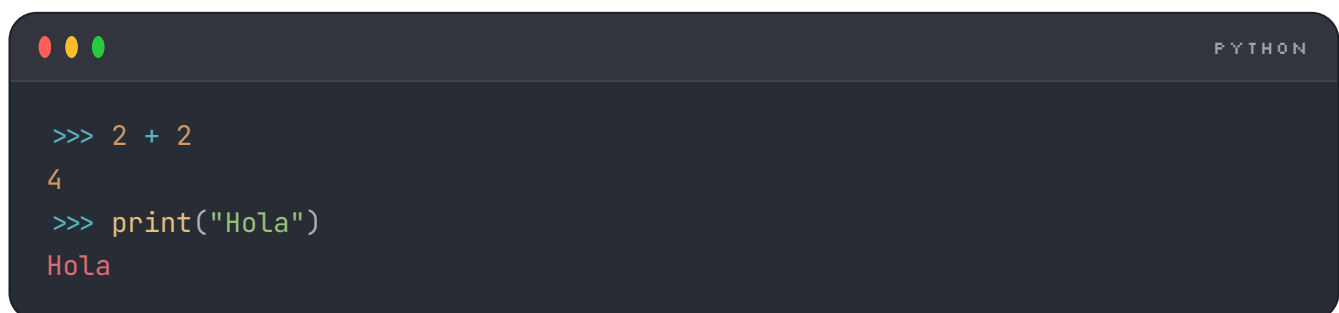
Pensá en Python como el **idioma de los Entrenadores**. Vos escribís instrucciones (el código) y la computadora las obedece como un Pokémon bien entrenado obedece a su dueño. Cuanto mejor escribas las órdenes, mejor pelea tu equipo.

3.2. ¿Cómo se corre Python?

Hay dos formas principales:

1. El REPL (modo interactivo)

Escribís `python3` en la terminal y se abre un intérprete donde probás cosas al instante:



```

PYTHON

>>> 2 + 2
4
>>> print("Hola")
Hola
  
```

El `>>>` es el prompt de Python. Es ideal para **experimentar**. Para salir, escribís `exit()`.

2. Archivos `.py`

Escribís tu programa en un archivo, por ejemplo `programa.py`, y lo corrés:



```

BASH

python3 programa.py
  
```



```
python3 programa.py
```

Así hacés programas de verdad que podés guardar y reusar.

3.3. `print()`: mostrar cosas en pantalla

`print()` muestra texto o valores en la pantalla. Es lo primero que hace todo programador.

```
print("¡Hola, mundo Pokémon!")  
print("Elegí a Pikachu como inicial")
```

Podés imprimir varias cosas separándolas con comas:

```
print("Mi Pokémon es", "Charizard") # Mi Pokémon es Charizard
```

3.4. Variables: cajas con nombre

Una **variable** guarda un dato con un nombre, para usarlo después. Es como una **Pokéball**: adentro guardás algo y le ponés una etiqueta.

```
nombre = "Pikachu" # guardamos el texto "Pikachu" en la variable nombre  
nivel = 25         # guardamos el número 25 en la variable nivel  
print(nombre)      # Pikachu  
print(nivel)       # 25
```

Regla: el nombre va a la izquierda del `=`, el valor a la derecha. **No** lleva espacios raros ni empieza con números.

```
pokemon_inicial = "Charmander" # bien (guiones bajos para separar palabras)
3pokemon = "no"                # mal (no puede empezar con número)
```

3.5. Tipos de datos

Cada dato tiene un **tipo**. Los cuatro básicos:

Tipo	Qué es	Ejemplo Pokémon
<code>int</code>	Número entero	nivel = <code>25</code> , hp = <code>100</code>
<code>float</code>	Número con decimales	peso = <code>6.0</code> , altura = <code>0.4</code>
<code>str</code>	Texto (string)	nombre = <code>"Pikachu"</code>
<code>bool</code>	Verdadero o falso	es_legendario = <code>True</code>

```
nivel = 25          # int
peso = 6.0          # float
nombre = "Pikachu"  # str (siempre entre comillas)
es_legendario = False # bool (True o False, con mayúscula)
```

Para ver el tipo de algo, usás `type()`:

```
print(type(nivel)) # <class 'int'>
print(type(nombre)) # <class 'str'>
```

3.6. input(): pedirle datos al usuario

`input()` **pausa** el programa y espera a que el usuario escriba algo y apriete Enter. Lo que escribe se guarda como texto.

```
nombre = input("¿Cómo se llama tu Pokémon? ")
print("¡Hola,", nombre + "!")
```

CUIDADO

MUY IMPORTANTE: `input()` SIEMPRE devuelve **texto (str)**, aunque el usuario escriba un número. Si querés un número, hay que convertirlo (lo vemos ahora).

3.7. Conversión de tipos

Para convertir entre tipos, usás funciones con el nombre del tipo:

```
nivel_texto = input("¿Nivel de tu Pokémon? ") # esto es str, ej "25"
nivel = int(nivel_texto)                     # ahora es int: 25
print(nivel + 5)                             # 30 (suma de números)
```

Funciones de conversión:

- `int("25")` → `25` (texto a entero)
- `float("6.5")` → `6.5` (texto a decimal)
- `str(25)` → `"25"` (número a texto)

```
# Forma compacta, muy común:
edad = int(input("¿Tu edad? ")) # pide y convierte en una sola línea
```

3.8. Comentarios

Un **comentario** es texto que Python ignora. Sirve para explicar tu código. Empieza con `#`.

```
# Esto es un comentario, Python no lo ejecuta.
nombre = "Pikachu" # también podés comentar al final de una línea
```

Comentar bien tu código es de buen programador. Tu yo del futuro te lo va a agradecer.

3.9. f-strings: la forma copada de armar texto

Un **f-string** te deja meter variables dentro de un texto poniendo una **f** antes de las comillas y las variables entre **{llaves}**.

```

nombre = "Pikachu"
nivel = 25

# Forma vieja (funciona pero es incómoda):
print("Mi " + nombre + " es nivel " + str(nivel))

# Forma copada con f-string:
print(f"Mi {nombre} es nivel {nivel}")    # Mi Pikachu es nivel 25

```

Los f-strings son **la forma recomendada**. Fijate que no hace falta convertir el número con **str()**: el f-string lo hace solo.

```

hp = 100
ataque = 55
print(f"{nombre} tiene {hp} HP y {ataque} de ataque")

```

3.10. Resumen de la semana

```

# Variables: caja con nombre
nombre = "Pikachu"

# Tipos: int, float, str, bool
nivel = 25          # int
peso = 6.0          # float
texto = "hola"      # str
activo = True       # bool

```

```
# print: mostrar en pantalla
print("Hola")

# input: pedir datos (devuelve SIEMPRE texto)
edad = input("Edad: ")

# Conversión de tipos
numero = int("25")
texto = str(25)

# f-string: armar texto con variables
print(f"{nombre} es nivel {nivel}")
```

Concepto	Para qué sirve
<code>print()</code>	Mostrar cosas
variable	Guardar un dato
<code>int/float/str/bool</code>	Los tipos de datos
<code>input()</code>	Pedir datos al usuario
<code>int()</code> , <code>str()</code>	Convertir tipos
<code>#</code>	Comentar
<code>f"... "</code>	Armar texto con variables

3.11. ¿Y ahora qué?

1. Hacé los ejercicios de `ejercicios.py` (escribí tu código donde dice `# TU CÓDIGO ACÁ`).
2. Compará con `soluciones.py`.
3. Corré los tests: `pytest semana-03-python-introduccion/`
4. Jugá el interactivo, que te genera tu **tarjeta de Entrenador**:

```
BASH

python interactivo.py
```

ÁNIMO

"El viaje de mil Pokémon empieza con un solo `print()`."

CAPÍTULO 4. Python: Control de Flujo

META

Meta de la semana: que tu programa **tome decisiones** y **repita** cosas. Hasta ahora tu código corría en línea recta. Ahora va a poder elegir caminos y dar vueltas.

4.1. Analogía: decisiones en batalla

En una batalla Pokémon, vos tomás decisiones todo el tiempo: "Si el rival es de tipo Agua, USO un ataque Eléctrico". Y repetís acciones: "MIENTRAS el rival tenga HP, sigo atacando".

Eso es exactamente el **control de flujo**: decirle al programa **cuándo** hacer algo (decisiones) y **cuántas veces** hacerlo (repeticiones).

4.2. if: tomar decisiones

`if` ejecuta un bloque de código **solo si** una condición es verdadera.

```
nivel = 30

if nivel ≥ 25:
    print("¡Tu Pokémon puede evolucionar!")
```

CUIDADO

La indentación importa. El código "adentro" del `if` va con **4 espacios** de sangría. Python usa esa sangría para saber qué está adentro y qué afuera. No es decorativo: es obligatorio.

4.3. Operadores de comparación

Sirven para construir condiciones. Devuelven `True` o `False`:

Operador	Significa	Ejemplo	Resultado
<code>=</code>	igual a	<code>nivel = 25</code>	True si nivel es 25
<code>≠</code>	distinto de	<code>tipo ≠ "Agua"</code>	True si no es Agua
<code>></code>	mayor que	<code>hp > 0</code>	True si hp es positivo
<code><</code>	menor que	<code>nivel < 100</code>	
<code>≥</code>	mayor o igual	<code>nivel ≥ 25</code>	
<code>≤</code>	menor o igual	<code>hp ≤ 0</code>	

CUIDADO

Ojo: `=` asigna (guarda en una variable), `==` compara. ¡No los confundas!

```

nivel = 25      # asignación: guardo 25 en nivel
nivel == 25     # comparación: ¿nivel es igual a 25? → True

```

4.4. elif y else: más caminos

`else` es el "si no". `elif` ("else if") agrega más condiciones a chequear.

```

hp = 40

if hp > 70:
    print("Tu Pokémon está sano ")
elif hp > 30:
    print("Tu Pokémon está cansado ")
elif hp > 0:
    print("¡Tu Pokémon está grave! ")
else:
    print("Tu Pokémon se debilitó ")

```


Python revisa las condiciones **de arriba hacia abajo** y entra en la **primera** que sea verdadera. Las demás las ignora.

4.5. Operadores lógicos: and, or, not

Combinan varias condiciones:

- `and` → verdadero si **ambas** son verdaderas.
- `or` → verdadero si **al menos una** es verdadera.
- `not` → invierte (verdadero ↔ falso).

```

nivel = 30
tipo = "Fuego"

# Evoluciona SI tiene nivel suficiente Y es de fuego.
if nivel ≥ 25 and tipo == "Fuego":
    print("¡Charmander evoluciona a Charmeleon!")

# Es fuerte SI es legendario O tiene nivel 100.
if es_legendario or nivel == 100:
    print("¡Pokémon poderoso!")

# Puede pelear SI NO está debilitado.
if not debilitado:
    print("¡A la batalla!")
  
```

4.6. while: repetir mientras...

`while` repite un bloque **mientras** una condición sea verdadera. Es el bucle de "seguí hasta que...".

```

hp_rival = 100

while hp_rival > 0:
    print(f"Atacás. HP del rival: {hp_rival}")
    hp_rival = hp_rival - 20 # bajamos el HP en cada vuelta

print("¡Ganaste la batalla! ")
  
```

CUIDADO

¡**Cuidado con los bucles infinitos!** Si la condición nunca se vuelve falsa, el programa nunca termina. Asegurate de que algo cambie adentro del `while` (acá, el HP baja).

4.7. for: repetir una cantidad de veces

`for` recorre una secuencia de valores, uno por uno. Es el bucle de "para cada...".



PYTHON

```
# Recorre una lista de Pokémon (las listas las vemos a fondo en la semana 6).
equipo = ["Pikachu", "Charizard", "Snorlax"]

for pokemon in equipo:
    print(f"Tengo a {pokemon}")
```

range(): generar números

`range()` genera una secuencia de números, ideal para repetir N veces.



PYTHON

```
# Repite 5 veces (del 0 al 4).
for i in range(5):
    print(f"Lanzamiento de Pokéball número {i + 1}")

# range(inicio, fin): del 1 al 10 (el 'fin' NO se incluye).
for nivel in range(1, 11):
    print(f"Nivel {nivel}")

# range(inicio, fin, paso): de 0 a 100 de 10 en 10.
for hp in range(0, 101, 10):
    print(hp)
```

TIP

Recordá: `range(5)` da `0, 1, 2, 3, 4` (empieza en 0, termina **antes** del 5).

4.8. break y continue

Controlan el bucle desde adentro:

- `break` → **corta** el bucle por completo.
- `continue` → **saltea** el resto de esta vuelta y pasa a la siguiente.



PYTHON

```
# break: dejamos de buscar al encontrar a Mewtwo.
for pokemon in equipo:
    if pokemon == "Mewtwo":
        print("¡Encontramos a Mewtwo!")
        break           # salimos del for inmediatamente

# continue: salteamos a los Pokémon debilitados.
for pokemon in equipo:
    if pokemon == "debilitado":
        continue        # saltamos a la próxima vuelta
    print(f"{pokemon} entra a la batalla")
```

4.9. Resumen de la semana



PYTHON

```
# if / elif / else: decisiones
if hp > 50:
    print("sano")
elif hp > 0:
    print("herido")
else:
    print("debilitado")

# Comparadores: ==, !=, >, <, >=, <=
# Lógicos: and, or, not
if nivel >= 25 and tipo == "Fuego":
    print("evoluciona")

# while: repetir mientras una condición sea verdadera
while hp > 0:
    hp -= 20

# for + range: repetir N veces
```

```
for i in range(5):
    print(i)

# break corta el bucle; continue saltea una vuelta
```

Concepto	Para qué sirve
<code>if/elif/else</code>	Elegir qué código correr
<code>=, !=, >, <</code>	Comparar valores
<code>and, or, not</code>	Combinar condiciones
<code>while</code>	Repetir mientras algo sea cierto
<code>for</code> + <code>range()</code>	Repetir una cantidad de veces
<code>break</code>	Cortar el bucle
<code>continue</code>	Saltar a la próxima vuelta

4.10. ¿Y ahora qué?

1. Resolvé `ejercicios.py`.
2. Corré los tests: `pytest semana-04-python-control-de-flujo/`
3. Jugá el **simulador de batalla por turnos**:

```
BASH
python interactivo.py
```

ÁNIMO

"En la batalla y en el código, las buenas decisiones ganan combates."

CAPÍTULO 5. Python: Funciones

META

Meta de la semana: aprender a empaquetar código en **funciones** para reusarlo, organizarlo y no repetirlo. Es uno de los conceptos más importantes de toda la programación.

5.1. Analogía: los ataques aprendidos

Un Pokémon aprende un ataque **una vez** y lo puede usar **mil veces**. No tiene que reaprender "Impactrueno" en cada batalla: lo sabe, le pone un nombre, y lo invoca cuando quiere.

Una **función** es eso: un bloque de código que escribís **una vez**, le ponés un **nombre**, y lo **llamás** (lo usás) todas las veces que quieras.

5.2. Definir una función con def



PYTHON

```
def saludar():
    print("¡Hola, Entrenador!")
```

Desglose:

- `def` → palabra clave para **definir** una función.
- `saludar` → el nombre que le ponés.
- `()` → los paréntesis (después vemos qué va adentro).
- `:` → dos puntos, obligatorios.
- El cuerpo va **indentado** (4 espacios), igual que en los `if`.

Definir una función no la ejecuta. Para ejecutarla, la **llamás** por su nombre:



PYTHON

```
saludar()    # ¡Hola, Entrenador!
saludar()    # ¡Hola, Entrenador! (la usamos de nuevo, gratis)
```

5.3. Parámetros: darle datos a la función

Un **parámetro** es un dato que la función recibe para trabajar. Va dentro de los paréntesis.

```
def saludar(nombre):
    print(f"¡Hola, {nombre}!")

saludar("Ash")      # ¡Hola, Ash!
saludar("Misty")    # ¡Hola, Misty!
```

Podés tener varios parámetros, separados por comas:

```
def presentar(nombre, tipo):
    print(f"{nombre} es de tipo {tipo}")

presentar("Pikachu", "Eléctrico")
```

TIP

"Parámetro" es el nombre en la definición (`nombre`). "Argumento" es el valor que pasás al llamar (`"Ash"`). Mucha gente usa los términos como sinónimos, no te preocupes por eso.

5.4. return: devolver un resultado

`return` hace que la función **devuelva** un valor para usarlo después. Es la diferencia entre una función que solo "muestra" y una que "calcula y entrega".

```
def calcular_dano(ataque, defensa):
    return ataque - defensa

# Guardamos lo que devuelve en una variable.
resultado = calcular_dano(50, 20)
print(resultado)    # 30
```

```
# 0 lo usamos directamente.
print(calcular_dano(80, 30)) # 50
```

CUIDADO

Cuando Python ejecuta un `return`, **sale** de la función inmediatamente. El código que esté después del `return` no se ejecuta.

print vs return — la confusión clásica

```
def con_print(x):
    print(x * 2)          # MUESTRA en pantalla, pero no devuelve nada

def con_return(x):
    return x * 2          # DEVUELVE el valor para seguir usándolo

a = con_print(5)          # imprime 10, pero a queda en None
b = con_return(5)         # no imprime nada, pero b vale 10
print(b + 1)              # 11 (porque b es un número de verdad)
```

Regla práctica: si querés **usar** el resultado después, usá `return`.

5.5. Valores por defecto

Podés darle a un parámetro un valor por defecto, que se usa si no pasás nada:

```
def atacar(nombre, dano=10):          # dano vale 10 si no se especifica
    print(f"{nombre} hizo {dano} de daño")

atacar("Pikachu")                    # Pikachu hizo 10 de daño (usa el default)
atacar("Charizard", 35)              # Charizard hizo 35 de daño (lo pisamos)
```

5.6. Scope: dónde vive cada variable

El **scope** (alcance) es la zona donde una variable existe. Una variable creada **adentro** de una función **solo existe ahí adentro**. Es como un Pokémon dentro de su Pokéball: afuera no se lo ve.

```
PYTHON

def entrenar():
    secreto = "técnica oculta"    # variable LOCAL: solo vive acá adentro
    print(secreto)

entrenar()
# print(secreto)    # ERROR: 'secreto' no existe afuera de la función
```

Las variables creadas afuera son **globales** y se pueden leer adentro:

```
PYTHON

entrenador = "Ash"    # variable global

def saludar():
    print(f"Hola {entrenador}")    # puede leer la global

saludar()    # Hola Ash
```

5.7. Funciones lambda: funciones express

Una **lambda** es una función chiquita de una sola línea, sin nombre. Útil para cosas simples.

```
PYTHON

# Función normal:
def doble(x):
    return x * 2

# La misma como lambda:
doble = lambda x: x * 2

print(doble(5))    # 10
```

La estructura es: `(lambda parámetros: expresión)`. El resultado de la expresión es lo que devuelve.



PYTHON

```
sumar = lambda a, b: a + b
print(sumar(3, 4))    # 7
```

TIP

Las lambda se usan mucho para ordenar o filtrar (lo vas a ver en la semana 6). Por ahora, alcanza con saber qué son.

5.8. Docstrings: documentar tu función

Un **docstring** es un texto (entre `"""triple comillas"""`) justo abajo del `def`, que explica qué hace la función.



PYTHON

```
def calcular_dano(ataque, defensa):
    """Calcula el daño restando la defensa al ataque."""
    return ataque - defensa
```

Sirve para que vos (y otros) entiendan la función sin leer todo el código. Es de buena programadora documentar así.

5.9. Recursión: una función que se llama a sí misma

Recursión es cuando una función se llama a sí misma para resolver un problema más chico. Suena raro, pero es potente.

El ejemplo clásico es el **factorial** ($5! = 5 \times 4 \times 3 \times 2 \times 1$):



PYTHON

```
def factorial(n):
    # Caso base: el punto donde la recursión FRENA.
    if n <= 1:
        return 1
    # Caso recursivo: la función se llama a sí misma con un número más chico.
    return n * factorial(n - 1)
```

```
print(factorial(5))    # 120
```

CUIDADO

Toda recursión necesita un **caso base** (una condición que la frene). Sin él, se llama infinitamente y revienta. Es como las muñecas rusas: en algún momento llegás a la más chiquita.

5.10. Resumen de la semana

```
# Definir
def atacar(nombre, dano=10):
    """Muestra un ataque (docstring)."""
    return f"{nombre} hizo {dano} de daño"

# Llamar
mensaje = atacar("Pikachu", 25)

# return devuelve un valor; print solo muestra
# Valores por defecto: dano=10
# Scope: las variables locales no existen afuera
# Lambda: función express
doble = lambda x: x * 2
# Recursión: función que se llama a sí misma (necesita caso base)
```

Concepto	Para qué sirve
<code>def</code>	Definir una función
parámetros	Pasarle datos
<code>return</code>	Devolver un resultado
valor por defecto	Parámetro opcional
scope	Dónde vive una variable
<code>Lambda</code>	Función chica de una línea

Concepto	Para qué sirve
docstring	Documentar qué hace
recursión	Función que se llama a sí misma

5.11. ¿Y ahora qué?

1. Resolvé `ejercicios.py` (varios te piden refactorizar código repetido en funciones).
2. Corré los tests: `pytest semana-05-python-funciones/`
3. Jugá la **calculadora de estadísticas Pokémon**:

```
python interactivo.py
```

ÁNIMO

"Una buena función es como un buen ataque: la aprendés una vez y te sirve toda la vida."

CAPÍTULO 6. Python: Listas y Colecciones

META

Meta de la semana: guardar **muchos** datos juntos. Hasta ahora cada variable guardaba UNA cosa. Ahora vas a manejar equipos enteros, Pokédex completas y mucho más.

6.1. Analogía: tu equipo Pokémon

Un Entrenador no lleva un solo Pokémon: lleva un **equipo** (hasta 6). Una **lista** es justamente eso: una colección ordenada de cosas guardadas bajo un solo nombre.

6.2. Listas: colección ordenada y modificable

Una **lista** se escribe entre corchetes `[]`, con los elementos separados por comas.



PYTHON

```
equipo = ["Pikachu", "Charizard", "Snorlax"]
niveles = [25, 50, 40]
mezcla = ["Pikachu", 25, True]    # puede tener tipos distintos
vacía = []                       # una lista vacía
```

Acceder por índice

Cada elemento tiene una **posición (índice)**. ¡Empieza en 0!



PYTHON

```
equipo = ["Pikachu", "Charizard", "Snorlax"]
print(equipo[0])    # Pikachu (el primero)
print(equipo[1])    # Charizard
print(equipo[-1])   # Snorlax (el último, contando desde atrás)
```

Modificar



PYTHON

```
equipo[0] = "Raichu"      # cambiamos el primero (Pikachu evolucionó)
print(equipo)             # ['Raichu', 'Charizard', 'Snorlax']
```

6.3. Métodos de listas (las acciones de tu equipo)

```
equipo = ["Pikachu", "Charizard"]

equipo.append("Snorlax")    # agrega al final →
['Pikachu', 'Charizard', 'Snorlax']
equipo.insert(0, "Mewtwo")  # inserta en una posición
equipo.remove("Charizard")  # borra por valor
ultimo = equipo.pop()       # saca y devuelve el último
print(len(equipo))          # cantidad de elementos
print("Pikachu" in equipo)  # ¿está? → True o False
equipo.sort()               # ordena la lista
equipo.reverse()            # la da vuelta
```

Método	Qué hace
<code>.append(x)</code>	Agrega x al final
<code>.insert(i, x)</code>	Inserta x en la posición i
<code>.remove(x)</code>	Borra la primera aparición de x
<code>.pop()</code>	Saca y devuelve el último
<code>len(lista)</code>	Cantidad de elementos
<code>x in lista</code>	¿x está en la lista?
<code>.sort()</code>	Ordena
<code>.reverse()</code>	Invierte el orden

6.4. Tuplas: colección que NO se modifica

Una **tupla** es como una lista, pero **inmutable** (no se puede cambiar). Se escribe con paréntesis `()`. Sirve para datos fijos.

```

coordenada = (10, 20)          # una posición que no debería cambiar
tipos_pikachu = ("Electrico",) # tupla de un solo elemento (ojo a la coma)

print(coordenada[0])           # 10
# coordenada[0] = 5           # ERROR: una tupla no se puede modificar

```

TIP

Usá tupla cuando los datos NO deberían cambiar (como las coordenadas de un punto). Usá lista cuando sí (como tu equipo, que cambia).

6.5. Sets: colección SIN duplicados

Un **set** (conjunto) guarda elementos **únicos**, sin orden y sin repetidos. Se escribe con llaves `{}`.

```

tipos_vistos = {"Fuego", "Agua", "Fuego", "Planta"}
print(tipos_vistos) # {'Fuego', 'Agua', 'Planta'} (el Fuego repetido
                    # desaparece)

tipos_vistos.add("Electrico") # agregar
print("Agua" in tipos_vistos) # True

```

Útil para sacar duplicados de una lista:

```

capturados = ["Pikachu", "Pidgey", "Pikachu", "Rattata"]
unicos = set(capturados)
print(len(unicos)) # 3 (sin contar el Pikachu repetido)

```

6.6. Diccionarios: pares clave → valor

Un **diccionario** guarda pares de **clave: valor**. Es como una Pokédex de verdad: buscás por nombre y obtenés los datos. Se escribe con llaves `{}`.

```
PYTHON

pikachu = {
    "nombre": "Pikachu",
    "tipo": "Electrico",
    "nivel": 25,
    "hp": 100,
}

print(pikachu["nombre"])    # Pikachu (accedés por la clave)
print(pikachu["nivel"])    # 25
```

Modificar y agregar

```
PYTHON

pikachu["nivel"] = 26        # cambiar un valor
pikachu["ataque"] = 55       # agregar una clave nueva
print(pikachu.get("defensa", 0))
# .get() devuelve un default si la clave no está
```

Recorrer un diccionario

```
PYTHON

for clave in pikachu:
    print(clave, "→", pikachu[clave])

# O directamente clave y valor con .items()
for clave, valor in pikachu.items():
    print(f"{clave}: {valor}")
```

Operación	Cómo
Acceder	<code>dic["clave"]</code>
Acceso seguro	<code>dic.get("clave", default)</code>

Operación	Cómo
Agregar/cambiar	<code>dic["clave"] = valor</code>
Borrar	<code>del dic["clave"]</code>
¿Existe la clave?	<code>"clave" in dic</code>
Recorrer	<code>for k, v in dic.items():</code>

6.7. Comprensiones de listas: crear listas en una línea

Una **comprensión** arma una lista nueva a partir de otra, en una sola línea. Es elegante y muy "pythónico".

```

niveles = [25, 50, 40, 15]

# Forma larga:
dobles = []
for n in niveles:
    dobles.append(n * 2)

# Comprensión (la misma cosa, en una línea):
dobles = [n * 2 for n in niveles]
print(dobles)    # [50, 100, 80, 30]

# Con condición (filtrar):
altos = [n for n in niveles if n ≥ 40]
print(altos)     # [50, 40]

```

Estructura: `[expresión for elemento in coleccion if condición]`.

6.8. enumerate y zip

enumerate: índice + valor a la vez

```

equipo = ["Pikachu", "Charizard", "Snorlax"]

```



```
for indice, pokemon in enumerate(equipo):
    print(f"{indice + 1}. {pokemon}")
# 1. Pikachu
# 2. Charizard
# 3. Snorlax
```

zip: recorrer dos listas en paralelo

```
nombres = ["Pikachu", "Charizard"]
niveles = [25, 50]

for nombre, nivel in zip(nombres, niveles):
    print(f"{nombre} es nivel {nivel}")
# Pikachu es nivel 25
# Charizard es nivel 50
```

6.9. Resumen de la semana

```
# Lista: ordenada y modificable
equipo = ["Pikachu", "Charizard"]
equipo.append("Snorlax")

# Tupla: inmutable
punto = (10, 20)

# Set: sin duplicados
tipos = {"Fuego", "Agua"}

# Diccionario: clave → valor
pikachu = {"nombre": "Pikachu", "nivel": 25}

# Comprensión: lista en una línea
dobles = [n * 2 for n in [1, 2, 3]]

# enumerate y zip
for i, p in enumerate(equipo): ...
for n, l in zip(nombres, niveles): ...
```

Colección	Símbolo	Característica
Lista	<code>[]</code>	Ordenada, modificable
Tupla	<code>()</code>	Ordenada, inmutable
Set	<code>{}</code>	Única, sin orden
Diccionario	<code>{clave: valor}</code>	Pares clave-valor

6.10. ¿Y ahora qué?

1. Resolvé `ejercicios.py` (énfasis en manipular datos).
2. Corré los tests: `pytest semana-06-python-listas-y-colecciones/`
3. Jugá el **gestor de equipo Pokémon**:

```
python interactivo.py
```

ÁNIMO

"Un buen Entrenador conoce a cada miembro de su equipo. Un buen programador conoce sus estructuras de datos."

CAPÍTULO 7. Python: Cadenas y Archivos

META

Meta de la semana: trabajar con texto en profundidad y hacer que tus datos **sobrevivan** cuando cerrás el programa (guardándolos en archivos).

7.1. Analogía: la Pokédex que recuerda

Hasta ahora, cuando cerrabas tu programa, todo se borraba: tu equipo, tus capturas, todo. Era una Pokédex con amnesia.

Esta semana le damos **memoria permanente**: vas a guardar datos en **archivos** del disco. Apagás la compu, volvés mañana, y tu Pokédex se acuerda de todo.

7.2. Métodos de strings (cadenas de texto)

Un string tiene un montón de **métodos** (acciones) para transformarlo. No modifican el original: **devuelven uno nuevo**.

```
nombre = "  Pikachu  "

print(nombre.upper())      # "  PIKACHU  "    (mayúsculas)
print(nombre.lower())      # "  pikachu  "    (minúsculas)
print(nombre.strip())      # "Pikachu"        (saca espacios de los bordes)
print(nombre.replace("a", "@")) # reemplaza
print("pikachu".capitalize()) # "Pikachu"    (primera en mayúscula)
print("Pikachu".startswith("Pika")) # True
print("Pikachu".endswith("chu"))   # True
print("Pikachu".find("ka"))        # 2      (posición donde aparece)
print(len("Pikachu"))              # 7      (longitud)
```

split y join: separar y unir

```
# split: parte un texto en una lista, usando un separador.
línea = "Pikachu,Electrico,25"
```

```
partes = linea.split(",")
print(partes)      # ['Pikachu', 'Electrico', '25']

# join: une una lista en un texto, con un separador.
datos = ["Charizard", "Fuego", "50"]
linea = ",".join(datos)
print(linea)       # "Charizard,Fuego,50"
```

TIP

`split` y `join` son **clave** para trabajar con archivos CSV (lo vemos abajo).

7.3. Slicing: rebanar strings

El **slicing** saca una "rebanada" de un string usando `[inicio:fin]`. El `fin` NO se incluye.

```
texto = "Pikachu"
#      0123456      (índices)

print(texto[0])      # "P"      (un carácter)
print(texto[0:4])    # "Pika"   (del 0 al 3)
print(texto[4:])     # "chu"    (del 4 hasta el final)
print(texto[:4])     # "Pika"   (del principio al 3)
print(texto[-3:])    # "chu"    (los últimos 3)
print(texto[::-1])   # "uhcakiP" (al revés)
```

7.4. Abrir archivos con open()

`open()` abre un archivo. Le pasás el nombre y el **modo**:

Modo	Significa
"r"	leer (read) — el archivo debe existir
"w"	escribir (write) — crea o REEMPLAZA todo
"a"	agregar (append) — escribe al final



PYTHON

```
# Escribir (¡pisa lo que había!)
archivo = open("equipo.txt", "w")
archivo.write("Pikachu\n")      # \n = salto de línea
archivo.write("Charizard\n")
archivo.close()                 # ¡siempre hay que cerrar!
```

7.5. with: la forma correcta de abrir archivos

El problema de `open()` solo es que hay que acordarse de `close()`. La forma recomendada es usar `with`, que cierra el archivo solo, aunque haya un error.



PYTHON

```
# Escribir
with open("equipo.txt", "w") as archivo:
    archivo.write("Pikachu\n")
    archivo.write("Charizard\n")
# Al salir del 'with', el archivo se cierra automáticamente.

# Leer todo de una
with open("equipo.txt", "r") as archivo:
    contenido = archivo.read()
    print(contenido)

# Leer línea por línea (lo más común)
with open("equipo.txt", "r") as archivo:
    for linea in archivo:
        print(linea.strip()) # strip saca el \n del final
```

TIP

Usá **siempre** `with`. Es más corto, más seguro, y es lo que vas a ver en todos lados.

7.6. CSV: datos en filas y columnas

Un **CSV** (Comma-Separated Values) es un archivo de texto donde cada línea es una fila y los datos van separados por comas. Es como una mini planilla de Excel.



PYTHON

```
nombre, tipo, nivel
Pikachu, Electrico, 25
Charizard, Fuego, 50
```

Lo podés manejar con `split` y `join`, o con el módulo `csv` de la librería estándar:



PYTHON

```
import csv

# Escribir un CSV
with open("pokedex.csv", "w", newline="", encoding="utf-8") as f:
    escritor = csv.writer(f)
    escritor.writerow(["nombre", "tipo", "nivel"]) # encabezado
    escritor.writerow(["Pikachu", "Electrico", 25])
    escritor.writerow(["Charizard", "Fuego", 50])

# Leer un CSV
with open("pokedex.csv", "r", encoding="utf-8") as f:
    lector = csv.reader(f)
    for fila in lector:
        print(fila) # cada fila es una lista: ['Pikachu', 'Electrico', '25']
```

TIP

`newline=""` y `encoding="utf-8"` son detalles que evitan problemas. Copialos siempre que uses CSV.

7.7. Manejo de excepciones: try / except

Un **error** (excepción) puede romper tu programa: abrir un archivo que no existe, convertir "abc" a número, etc. Con `try` / `except` lo **atrapás** y seguís adelante.



PYTHON

```
try:
    # Código que PODRÍA fallar.
```

```

numero = int(input("Nivel: "))
print(f"El nivel es {numero}")
except ValueError:
    # Esto corre SOLO si hubo un ValueError.
    print("Eso no es un número válido")

```

Atrapar errores al abrir archivos:

```

try:
    with open("noexiste.txt", "r") as f:
        contenido = f.read()
except FileNotFoundError:
    print("El archivo no existe, creando uno nuevo...")
    contenido = ""

```

Errores comunes que vas a atrapar:

- `ValueError` → conversión inválida (`int("abc")`).
- `FileNotFoundError` → archivo que no existe.
- `KeyError` → clave que no está en un diccionario.
- `ZeroDivisionError` → división por cero.

TIP

Atrapa **errores específicos** (como `ValueError`), no un `except:` pelado. Así sabés qué estás manejando.

7.8. Resumen de la semana

```

# Métodos de string
"  Hola  ".strip().upper()      # "HOLA"
"a,b,c".split(",")              # ['a', 'b', 'c']
", ".join(["a", "b"])           # "a,b"

# Slicing
"Pikachu"[0:4]                  # "Pika"

```

```
# Archivos con with
with open("datos.txt", "w") as f:
    f.write("hola\n")

with open("datos.txt", "r") as f:
    for linea in f:
        print(linea.strip())

# CSV con la librería estándar
import csv
# csv.writer / csv.reader

# Manejo de errores
try:
    n = int("abc")
except ValueError:
    print("no es número")
```

Concepto	Para qué sirve
métodos de string	Transformar texto
<code>split</code> / <code>join</code>	Separar / unir texto
slicing <code>[i:j]</code>	Rebanar texto
<code>with open()</code>	Abrir archivos seguro
módulo <code>csv</code>	Datos en filas/columnas
<code>try</code> / <code>except</code>	Atrapar errores

7.9. ¿Y ahora qué?

1. Resolvé `ejercicios.py`.
2. Corré los tests: `pytest semana-07-python-cadenas-y-archivos/`
3. Mirá los datos de ejemplo en la carpeta `data/`.
4. Jugá la **Pokédex con persistencia** (¡guarda y carga datos!):




```
python interactivo.py
```

ÁNIMO

"Los datos que no guardás, se los lleva el viento. Una buena Pokédex nunca olvida."

CAPÍTULO 8. Git: Control de Versiones

NOTA

Semana relajada. Después de tanto Python, frenamos un toque para aprender **Git**: la herramienta que usan TODOS los programadores para guardar su trabajo. Es fácil, es útil, y te va a salvar la vida mil veces.

8.1. Analogía: Git es "guardar la partida"

¿Te acordás cuando jugás Pokémon y **guardás la partida** antes de una pelea difícil? Si perdés, recargás y volvés al punto de guardado.

Git es exactamente eso, pero para tu código. Cada vez que hacés un "guardado" (se llama **commit**), Git recuerda cómo estaba TODO tu proyecto en ese momento. ¿Rompió algo? Volvés a un guardado anterior. ¿Querés probar una idea loca sin arruinar lo que funciona? Creás una **línea temporal alternativa** (una branch).

En Pokémon	En Git
Guardar la partida	Hacer un commit
El cartucho/partida	El repositorio
La PC de Bill (en la nube)	GitHub
Subir Pokémon a la PC	push
Bajar Pokémon de la PC	pull
Copiar la partida de un amigo	clone
Un universo paralelo (Ultraentes)	Una branch (rama)

8.2. ¿Por qué usar Git?

- **Nunca más perdés tu trabajo.** Cada commit es un punto seguro.
- **Volvés atrás** cuando rompés algo (y vas a romper cosas, es normal).
- **Probás ideas** sin miedo, en ramas separadas.

- **Trabajás con otros** sin pisarse el código.
 - **Guardás todo en GitHub**, accesible desde cualquier lado.
 - Todo programador lo usa. Saberlo es **obligatorio** en el mundo real.
-

8.3. Configuración inicial (una sola vez)

Antes de usar Git, le decís quién sos (queda en tus commits):

```
BASH
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
```

Esto se hace **una sola vez** en tu computadora.

8.4. Crear un repositorio: git init

Un **repositorio** (o "repo") es una carpeta que Git está vigilando. Para empezar a vigilar una carpeta:

```
BASH
cd mi-proyecto
git init
```

Esto crea una carpeta oculta `.git/` donde Git guarda toda la historia. **No la toques**, es la "memoria" de Git.

8.5. Ver el estado: git status

`git status` es tu mejor amigo. Te dice qué cambió, qué está listo para guardar y qué no. **Usalo todo el tiempo.**

```
BASH
git status
```

Te muestra cosas como:

- **Untracked files:** archivos nuevos que Git todavía no sigue.

- **Changes to be committed:** cambios listos para el próximo commit (en el "staging").
- **Changes not staged:** cambios que hiciste pero todavía no preparaste.

8.6. Los 3 pasos de un guardado

Guardar en Git tiene 3 zonas, como preparar tu equipo antes de una batalla:



1. `git add` — preparar lo que vas a guardar

```

git add pokemon.txt      # prepara un archivo
git add .                # prepara TODOS los cambios (el punto = "todo")
  
```

`git add` mueve tus cambios al **staging area**: "esto quiero guardarlo".

2. `git commit` — guardar la partida

```

git commit -m "Agregué a Pikachu al equipo"
  
```

`commit` crea el punto de guardado. El `-m` es el **mensaje**: una descripción de QUÉ hiciste. Escribí mensajes claros, tu yo del futuro te lo agradece.

TIP

Regla de oro: `git add` → `git commit`. Primero prepararás, después guardás.

8.7. Ver la historia: git log

`git log` muestra todos tus commits, del más nuevo al más viejo:

```

git log
git log --oneline      # versión cortita, una línea por commit

```

Cada commit tiene un código único (un "hash") y tu mensaje. Es la lista de todos tus puntos de guardado.

8.8. Ramas (branches): líneas temporales alternativas

Una **branch** (rama) es una **línea temporal paralela** de tu proyecto. Te deja probar cosas sin tocar lo que ya funciona. Es como entrar a un universo alternativo: si sale mal, volvé al principal y no pasó nada.

```

git branch                # muestra las ramas (la actual tiene un *)
git branch nueva-aventura # crea una rama nueva
git switch nueva-aventura # te cambiás a esa rama
git switch -c otra-rama   # crea Y se cambia, todo junto

```

TIP

`git checkout nueva-aventura` hace lo mismo que `git switch`. `switch` es la forma nueva y más clara; vas a ver las dos por ahí.

La rama principal se suele llamar `main` (antes se llamaba `master`).

```

      o---o---o  nueva-aventura  ← probás cosas acá
      /
o---o---o---o  main              ← lo estable queda intacto acá

```

8.9. Fusionar ramas: git merge

Cuando tu experimento en la rama funcionó, lo **fusionás** de vuelta a `main`:



B A S H

```
git switch main          # te parás en la rama destino
git merge nueva-aventura # traés los cambios de la otra rama
```

Las dos líneas temporales se unen en una.

8.10. GitHub: la PC de Bill en la nube

GitHub es un sitio web donde guardás tus repos en internet. Es como subir tus Pokémon a la **PC de Bill**: quedan seguros y los podés bajar desde cualquier computadora.

Conectar tu repo con GitHub



B A S H

```
git remote add origin https://github.com/usuario/repo.git
```

origin es el "apodo" de tu repo en GitHub.

Subir cambios: git push



B A S H

```
git push -u origin main # la primera vez (-u recuerda la conexión)
git push                 # las siguientes veces, alcanza con esto
```

Bajar cambios: git pull



B A S H

```
git pull # trae los cambios nuevos de GitHub
```

Copiar un repo de internet: git clone



B A S H

```
git clone https://github.com/usuario/repo.git
```

clone te baja una copia completa de un repo (con toda su historia). Así empezaste vos este curso.

8.11. .gitignore: lo que Git debe ignorar

A veces hay archivos que **no** querés guardar (datos temporales, contraseñas, el `venv`). Los listás en un archivo llamado `.gitignore` y Git los ignora:

```
# .gitignore
venv/
__pycache__/
*.db
progreso.json
```

TIP

Este mismo curso tiene un `.gitignore`. Abrilo y mirá qué ignora.

8.12. El flujo de trabajo típico (memorizalo)

```
# 1. Trabajás, editás archivos...
# 2. Ves qué cambió
git status
# 3. Preparás los cambios
git add .
# 4. Guardás con un mensaje
git commit -m "Descripción de lo que hice"
# 5. Subís a GitHub
git push
```

Este ciclo lo vas a repetir **miles** de veces en tu vida de programador.

8.13. Resumen de la semana

Comando	Qué hace
<code>git init</code>	Empieza a vigilar una carpeta
<code>git status</code>	Muestra qué cambió
<code>git add <archivo></code>	Prepara cambios (staging)
<code>git add .</code>	Prepara TODOS los cambios
<code>git commit -m "msg"</code>	Guarda la partida (punto de guardado)
<code>git log</code>	Muestra la historia de commits
<code>git branch <nombre></code>	Crea una rama
<code>git switch <nombre></code>	Cambia de rama
<code>git switch -c <nombre></code>	Crea y cambia de rama
<code>git merge <rama></code>	Fusiona una rama en la actual
<code>git remote add origin <url></code>	Conecta con GitHub
<code>git push</code>	Sube a GitHub
<code>git pull</code>	Baja de GitHub
<code>git clone <url></code>	Copia un repo de internet

8.14. ¿Y ahora qué?

1. Hacé los ejercicios de `ejercicios.md` en una carpeta de prueba (¡con Git de verdad!).
2. Si te trabás, mirá `soluciones.md`.
3. Jugá el **simulador de Git** para practicar sin miedo:

```
python interactivo.py
```


ÁNIMO

"Programar sin Git es como jugar Pokémon sin poder guardar la partida. Una vez que lo usás, no querés volver atrás."

CAPÍTULO 9. Python: POO Introducción

META

Meta de la semana: aprender **Programación Orientada a Objetos (POO)**. Vas a crear tus propios "moldes" para fabricar Pokémon, cada uno con sus datos y sus acciones.

9.1. Analogía: el molde y los Pokémon

Imaginá la **especie "Pikachu"** como un **molde**: define que todo Pikachu tiene un nombre, un nivel, HP, y que puede usar Impactrueno. Pero TU Pikachu y el Pikachu de Ash son **individuos distintos**: mismo molde, datos diferentes.

En POO:

- El **molde** se llama **clase** (la especie, el plano).
- Cada **individuo** fabricado con ese molde se llama **objeto** o **instancia** (tu Pikachu concreto).

9.2. Definir una clase

Una **clase** se define con la palabra `class`. Por convención, su nombre empieza con **mayúscula**.

```
class Pokemon:
    pass    # por ahora vacía
```

Para **crear un objeto** (instancia) de esa clase, la llamás como si fuera una función:

```
mi_pokemon = Pokemon()    # fabricamos un Pokémon con el molde
```

9.3. `__init__`: el constructor

El método `__init__` es el **constructor**: se ejecuta automáticamente cuando creás un objeto. Sirve para darle sus datos iniciales.



PYTHON

```
class Pokemon:
    def __init__(self, nombre, tipo, nivel):
        # 'self' es el objeto que se está creando.
        # Guardamos los datos DENTRO del objeto con self.
        self.nombre = nombre
        self.tipo = tipo
        self.nivel = nivel

# Al crear el objeto, pasamos los datos que pide __init__.
pikachu = Pokemon("Pikachu", "Electrico", 25)
print(pikachu.nombre)    # Pikachu
print(pikachu.nivel)    # 25
```

¿Qué es `self`?

`self` es el **propio objeto**. Es la forma en que el objeto se refiere a sí mismo. Cuando hacés `self.nombre = nombre`, estás diciendo "guardá este nombre adentro mío".

TIP

`self` siempre es el **primer parámetro** de los métodos, pero **no lo pasás** al llamarlos: Python lo hace solo. Es automático.

9.4. Atributos: los datos del objeto

Los **atributos** son las variables que viven dentro de un objeto (`self.nombre`, `self.nivel`...). Son sus "stats".



PYTHON

```
pikachu = Pokemon("Pikachu", "Electrico", 25)

print(pikachu.nivel)    # leer un atributo: 25
pikachu.nivel = 26      # modificar un atributo
print(pikachu.nivel)    # 26
```

Cada objeto tiene **sus propios** atributos, independientes de los demás:



PYTHON

```
pikachu = Pokemon("Pikachu", "Electrico", 25)
charizard = Pokemon("Charizard", "Fuego", 50)

print(pikachu.nivel)      # 25
print(charizard.nivel)    # 50 (son distintos)
```

9.5. Métodos: las acciones del objeto

Un **método** es una función definida dentro de la clase. Son las "acciones" o "ataques" que el objeto puede hacer. Siempre reciben `self` primero.

```
class Pokemon:
    def __init__(self, nombre, tipo, nivel):
        self.nombre = nombre
        self.tipo = tipo
        self.nivel = nivel
        self.hp = 100

    def atacar(self):
        # Un método puede usar los atributos del objeto con self.
        return f"{self.nombre} ataca con un golpe de tipo {self.tipo}!"

    def subir_nivel(self):
        # Un método puede modificar los atributos del objeto.
        self.nivel = self.nivel + 1

pikachu = Pokemon("Pikachu", "Electrico", 25)
print(pikachu.atacar())      # Pikachu ataca con un golpe de tipo Electrico!
pikachu.subir_nivel()
print(pikachu.nivel)        # 26
```

TIP

Fíjate: llamas los métodos con `objeto.metodo()`. No pasas `self`, Python lo manda solo.

9.6. str: cómo se muestra el objeto

Por defecto, si hacés `print(pikachu)`, Python muestra algo feo como `<__main__.Pokemon object at 0x7f...>`. El método especial `__str__` define un texto lindo.

```
class Pokemon:
    def __init__(self, nombre, nivel):
        self.nombre = nombre
        self.nivel = nivel

    def __str__(self):
        # Lo que devuelva esto es lo que ve el usuario con print().
        return f"{self.nombre} (Nivel {self.nivel})"

pikachu = Pokemon("Pikachu", 25)
print(pikachu)      # Pikachu (Nivel 25) ← ¡mucho mejor!
```

9.7. repr: la versión para programadores

`__repr__` es parecido a `__str__`, pero es la representación "técnica", la que ves en el REPL o cuando un objeto está dentro de una lista. La idea es que sea precisa y útil para debuggear.

```
class Pokemon:
    def __init__(self, nombre, nivel):
        self.nombre = nombre
        self.nivel = nivel

    def __repr__(self):
        return f"Pokemon(nombre='{self.nombre}', nivel={self.nivel})"

pikachu = Pokemon("Pikachu", 25)
print(repr(pikachu))      # Pokemon(nombre='Pikachu', nivel=25)
equipo = [pikachu]
print(equipo)             # [Pokemon(nombre='Pikachu', nivel=25)]
```

TIP

Regla práctica: `__str__` para el usuario (lindo), `__repr__` para vos (preciso). Si solo vas a definir uno, definí `__repr__`.

9.8. Encapsulamiento básico

Encapsular es proteger los datos internos de un objeto. En Python, por convención, un atributo "privado" (que no debería tocarse desde afuera) se nombra con un **guión bajo** adelante.

```
class Pokemon:
    def __init__(self, nombre):
        self.nombre = nombre
        self._hp = 100          # _hp: "esto es interno, no lo toques directo"

    def recibir_dano(self, cantidad):
        # La forma CORRECTA de cambiar el HP es a través de un método,
        # que puede validar (que no baje de 0, por ejemplo).
        self._hp = self._hp - cantidad
        if self._hp < 0:
            self._hp = 0

    def hp_actual(self):
        return self._hp
```

El guión bajo no lo "bloquea" de verdad (Python confía en vos), pero es una señal clara: "accedé a esto solo a través de los métodos". En la semana 9 lo profundizamos con `@property`.

9.9. Resumen de la semana

```
class Pokemon:                                # clase = molde
    def __init__(self, nombre, nivel):        # constructor
        self.nombre = nombre                 # atributos (datos del objeto)
        self.nivel = nivel
        self._hp = 100                       # _hp: atributo "interno"

    def atacar(self):                          # método (acción)
        return f"{self.nombre} ataca!"
```

```

def __str__(self):                # texto lindo para print()
    return f"{self.nombre} (Nv {self.nivel})"

def __repr__(self):              # texto técnico para debug
    return f"Pokemon({self.nombre!r}, {self.nivel})"

pikachu = Pokemon("Pikachu", 25) # instancia (objeto)
print(pikachu.atacar())
print(pikachu)

```

Concepto	Qué es
<code>class</code>	El molde / plano
objeto / instancia	Un individuo hecho con el molde
<code>__init__</code>	El constructor (datos iniciales)
<code>self</code>	El propio objeto
atributo	Un dato del objeto (<code>self.x</code>)
método	Una acción del objeto
<code>__str__</code>	Texto lindo para el usuario
<code>__repr__</code>	Texto técnico para debug
<code>_atributo</code>	Convención de "interno/privado"

9.10. ¿Y ahora qué?

1. Resolvé `ejercicios.py` (vas de una clase básica a una con métodos de batalla).
2. Corré los tests: `pytest semana-08-python-poo-introduccion/`
3. Jugá el **creador de Pokémon personalizado**:

```

python interactivo.py

```

ÁNIMO

"Una clase es un molde. Con un buen molde, fabricás Pokémon ilimitados."

CAPÍTULO 10. Python: POO Avanzado

META

Meta de la semana: llevar la POO al siguiente nivel con **herencia** y **polimorfismo**. Vas a crear familias de clases donde los Pokémon de Fuego, Agua y Planta comparten cosas pero cada uno tiene su estilo.

10.1. Analogía: las especies y los tipos

Todos los Pokémon comparten cosas básicas (nombre, HP, pueden atacar). Pero un **Charizard** (Fuego) y un **Blastoise** (Agua) atacan distinto y tienen ventajas diferentes.

La **herencia** te deja escribir lo común UNA vez (en una clase "padre") y que cada tipo lo **herede** y le agregue lo suyo.

10.2. Herencia: heredar de una clase padre

Una clase puede **heredar** de otra: recibe todos sus atributos y métodos, y puede agregar o cambiar lo que quiera.

```

# Clase PADRE (o "base", o "superclase")
class Pokemon:
    def __init__(self, nombre, nivel):
        self.nombre = nombre
        self.nivel = nivel

    def atacar(self):
        return f"{self.nombre} ataca!"

# Clase HIJA: hereda de Pokemon (se pone entre paréntesis)
class PokemonFuego(Pokemon):
    def lanzallamas(self):
        return f"{self.nombre} usa Lanzallamas! "

# El hijo tiene LO SUYO y LO HEREDADO:
charizard = PokemonFuego("Charizard", 50)

```

```
print(charizard.atacar())      # heredado de Pokemon
print(charizard.lanzallamas()) # propio de PokemonFuego
```

10.3. super(): llamar al padre

Cuando el hijo define su propio `__init__`, puede llamar al del padre con `super()` para no repetir código.

```
class Pokemon:
    def __init__(self, nombre, nivel):
        self.nombre = nombre
        self.nivel = nivel

class PokemonFuego(Pokemon):
    def __init__(self, nombre, nivel):
        # super() llama al __init__ del padre: setea nombre y nivel.
        super().__init__(nombre, nivel)
        # Después agregamos lo propio del tipo Fuego.
        self.tipo = "Fuego"
        self.poder_fuego = 80

charizard = PokemonFuego("Charizard", 50)
print(charizard.nombre)      # Charizard (lo seteo el padre)
print(charizard.tipo)       # Fuego (lo seteo el hijo)
```

10.4. Polimorfismo: cada uno responde a su manera

Polimorfismo ("muchas formas") significa que el mismo método se comporta distinto según la clase. Todos los Pokémon tienen `atacar()`, pero cada tipo lo **sobrescribe** a su manera.

```
class Pokemon:
    def __init__(self, nombre):
        self.nombre = nombre

    def atacar(self):
        return f"{self.nombre} usa un ataque normal"
```

```

class PokemonFuego(Pokemon):
    def atacar(self):          # SOBREScribe el método del padre
        return f"{self.nombre} usa Lanzallamas! "

class PokemonAgua(Pokemon):
    def atacar(self):
        return f"{self.nombre} usa Pistola Agua! "

# El mismo método atacar(), distinto resultado según el tipo:
equipo = [PokemonFuego("Charizard"), PokemonAgua("Blastoise")]
for p in equipo:
    print(p.atacar())
# Charizard usa Lanzallamas!
# Blastoise usa Pistola Agua!

```

TIP

Esto es potente: podés tratar a todos como "Pokémon" y cada uno hace lo suyo. No necesitás `if tipo == "fuego"` por todos lados.

10.5. Métodos de clase y estáticos

Método estático (`@staticmethod`)

Una función que vive dentro de la clase pero **no usa** `self`. Es una utilidad relacionada con la clase.

```

class Pokemon:
    @staticmethod
    def es_tipo_valido(tipo):
        return tipo in ["Fuego", "Agua", "Planta", "Electrico"]

# Se llama desde la clase, sin crear un objeto:
print(Pokemon.es_tipo_valido("Fuego"))    # True
print(Pokemon.es_tipo_valido("Pizza"))    # False

```

Método de clase (`@classmethod`)

Recibe la **clase** (`cls`) en vez del objeto. Útil para crear objetos de formas alternativas ("constructores extra").

```
class Pokemon:
    def __init__(self, nombre, nivel):
        self.nombre = nombre
        self.nivel = nivel

    @classmethod
    def recien_nacido(cls, nombre):
        # cls es la clase. Creamos un Pokémon nivel 1.
        return cls(nombre, 1)

bebe = Pokemon.recien_nacido("Pichu")
print(bebe.nivel)    # 1
```

10.6. Propiedades con `@property`

`@property` te deja usar un método **como si fuera un atributo** (sin paréntesis). Sirve para calcular valores o validar. Es la forma elegante del encapsulamiento.

```
class Pokemon:
    def __init__(self, nombre, hp):
        self.nombre = nombre
        self._hp = hp          # atributo interno

    @property
    def hp(self):
        # Se accede como pokemon.hp (sin paréntesis).
        return self._hp

    @hp.setter
    def hp(self, valor):
        # Se ejecuta al hacer pokemon.hp = algo. Validamos acá.
        if valor < 0:
            valor = 0
        self._hp = valor
```

```
pikachu = Pokemon("Pikachu", 100)
print(pikachu.hp)      # 100 (parece atributo, pero pasa por el método)
pikachu.hp = -50       # el setter lo corrige
print(pikachu.hp)      # 0 (¡no quedó negativo!)
```

10.7. Clases abstractas con abc

Una **clase abstracta** es un molde que **no se puede instanciar directamente**: obliga a las clases hijas a implementar ciertos métodos. Se usa el módulo `abc`.

```
from abc import ABC, abstractmethod

class Pokemon(ABC):          # ABC = Abstract Base Class
    def __init__(self, nombre):
        self.nombre = nombre

    @abstractmethod
    def atacar(self):
        # No tiene cuerpo: cada hijo DEBE implementarlo.
        pass

# No se puede hacer Pokemon("X"): es abstracta.
# pokemon = Pokemon("X")    # ERROR

class PokemonFuego(Pokemon):
    def atacar(self):         # obligatorio implementarlo
        return f"{self.nombre} usa Lanzallamas!"

charizard = PokemonFuego("Charizard")  # funciona
print(charizard.atacar())
```

TIP

Las clases abstractas sirven para definir un "contrato": "todo Pokémon DEBE saber atacar". Si un hijo se olvida de implementar `atacar()`, Python no lo deja crear el objeto.

10.8. Resumen de la semana

```

from abc import ABC, abstractmethod

class Pokemon(ABC):
    # clase abstracta
    def __init__(self, nombre, nivel):
        self.nombre = nombre
        self._hp = 100

    @property
    # atributo calculado/validado
    def hp(self):
        return self._hp

    @hp.setter
    def hp(self, valor):
        self._hp = max(0, valor)

    @staticmethod
    # utilidad sin self
    def es_tipo_valido(tipo):
        return tipo in ["Fuego", "Agua"]

    @abstractmethod
    # cada hijo DEBE implementarlo
    def atacar(self):
        pass

class PokemonFuego(Pokemon):
    # herencia
    def __init__(self, nombre, nivel):
        super().__init__(nombre, nivel)  # llama al padre
        self.tipo = "Fuego"

    def atacar(self):
        # polimorfismo (sobrescribe)
        return f"{self.nombre} usa Lanzallamas!"

```

Concepto	Qué es
herencia	Una clase hija recibe lo del padre
<code>super()</code>	Llama métodos del padre
polimorfismo	El mismo método, distinto según la clase

Concepto	Qué es
<code>@staticmethod</code>	Función en la clase sin <code>self</code>
<code>@classmethod</code>	Recibe la clase (<code>cls</code>)
<code>@property</code>	Método usado como atributo
<code>abc</code> / <code>@abstractmethod</code>	Clase abstracta (contrato obligatorio)

10.9. ¿Y ahora qué?

1. Resolvé `ejercicios.py`.
2. Corré los tests: `pytest semana-09-python-poo-avanzado/`
3. Jugá el **sistema de tipos Pokémon** (batallas con ventajas):

```
python interactivo.py
```

ÁNIMO

"Un buen sistema de clases es como una buena familia Pokémon: comparten raíces, pero cada uno brilla a su manera."

CAPÍTULO 11. Python: Módulos y pip

META

Meta de la semana: usar código que ya escribieron otros. Vas a aprender a importar **módulos** de Python, instalar **librerías** con pip, y hasta consultar datos **reales de internet** con la PokéAPI.

11.1. Analogía: las MT (Máquinas Técnicas)

En los juegos Pokémon, las **MT** te enseñan ataques poderosos sin tener que entrenarlos desde cero. Las metés y listo, tu Pokémon ya sabe Rayo.

Los **módulos** y **librerías** son las MT del programador: código ya hecho que **importás** y usás. No reinventás la rueda: aprovechás el trabajo de millones de personas.

11.2. import: traer un módulo

Un **módulo** es un archivo de Python lleno de funciones útiles. Python trae muchos "de fábrica" (la **librería estándar**). Para usarlos, los importás.



PYTHON

```
import math

print(math.sqrt(16))    # 4.0   (raíz cuadrada)
print(math.pi)         # 3.141592653589793
print(math.ceil(4.2))  # 5     (redondea para arriba)
```

Formas de importar



PYTHON

```
import math                # importás todo el módulo, usás math.algo
from math import sqrt      # importás solo sqrt, lo usás directo
from math import sqrt, pi  # varios
import math as m           # le ponés un apodo: m.sqrt(16)
```


11.3. Módulos estándar útiles

`math` — matemática

```
import math
math.sqrt(25)      # 5.0
math.ceil(4.1)     # 5
math.floor(4.9)    # 4
math.pow(2, 3)     # 8.0
```

`random` — azar

```
import random
random.randint(1, 6)          # número al azar entre 1 y 6 (dado)
random.choice(["Pikachu", "Onix"]) # elige uno al azar de la lista
random.shuffle(equipo)        # mezcla la lista (la modifica)
```

`datetime` — fechas y horas

```
from datetime import datetime, date
date.today()          # la fecha de hoy
datetime.now()         # fecha y hora ahora
date.today().isoformat() # "2024-06-04" (texto estándar)
datetime.now().year    # el año actual
```

`os` y `sys` — sistema

```
import os
os.path.exists("datos.txt") # ¿existe el archivo? True/False
os.path.basename("/a/b/c.txt") # "c.txt" (solo el nombre)

import sys
```

```
sys.argv      # lista de argumentos pasados al programa
sys.exit()    # termina el programa
```

json — guardar/cargar datos estructurados

```
import json

datos = {"nombre": "Pikachu", "nivel": 25}

texto = json.dumps(datos)      # dict → texto JSON
print(texto)                   # {"nombre": "Pikachu", "nivel": 25}

recuperado = json.loads(texto) # texto JSON → dict
print(recuperado["nombre"])    # Pikachu

# Guardar y cargar archivos JSON
with open("pokemon.json", "w") as f:
    json.dump(datos, f)
with open("pokemon.json", "r") as f:
    datos = json.load(f)
```

TIP

JSON es **el formato** para guardar datos estructurados. Lo vas a usar en los proyectos finales. ¡Prestale atención!

11.4. Crear tu propio módulo

Cualquier archivo `.py` tuyo es un módulo que podés importar desde otro archivo. ¡Así organizás proyectos grandes!

Archivo `pokeutils.py`:

```
def formatear_nombre(nombre):
    return nombre.capitalize()
```

```
def es_legendario(nombre):
    return nombre.lower() in ["mewtwo", "mew", "articuno"]
```

Archivo `main.py` (en la misma carpeta):

```
import pokeutils

print(pokeutils.formatear_nombre("pikachu"))    # Pikachu
print(pokeutils.es_legendario("Mewtwo"))        # True
```

TIP

En esta semana hay un `pokeutils.py` de ejemplo. Miralo: así se ven los módulos propios.

11.5. pip: instalar librerías externas

La librería estándar es enorme, pero a veces necesitás algo que no viene. **pip** es el instalador de paquetes de Python: tu PokéMart de librerías.

```
pip install requests    # instala la librería 'requests'
pip install flask       # instala Flask (semana 12)
pip list                # muestra lo que tenés instalado
pip uninstall requests  # la desinstala
```

11.6. venv: entornos virtuales

Un **entorno virtual (venv)** es una "cajita" aislada con sus propias librerías, separada del Python del sistema. Así cada proyecto tiene sus versiones sin pisarse.

```
python3 -m venv venv    # crea el entorno en la carpeta 'venv'
source venv/bin/activate # lo activás (Linux/Mac)
```

```
pip install requests      # instala DENTRO del venv
deactivate                # lo desactivás
```

TIP

El `setup.sh` del curso hace esto por vos. Pero ahora entendés qué hace.

11.7. requirements.txt

Un archivo `requirements.txt` lista las librerías que tu proyecto necesita, para que cualquiera las instale de un saque:

```
requests ≥ 2.31
flask ≥ 3.0
```

Se instalan todas juntas con:

```
pip install -r requirements.txt
```

11.8. requests: consumir una API

Una **API** es un servicio en internet que te devuelve datos. La **PokéAPI** (<https://pokeapi.co>) tiene datos de TODOS los Pokémon, gratis.

La librería **requests** te deja pedir esos datos:

```
import requests

# Pedimos los datos de Pikachu.
respuesta = requests.get("https://pokeapi.co/api/v2/pokemon/pikachu")

# .json() convierte la respuesta a un diccionario de Python.
datos = respuesta.json()
```

```
print(datos["name"])           # pikachu
print(datos["height"])        # altura
print(datos["weight"])        # peso
print(datos["types"][0]["type"]["name"])# electric
```

CUIDADO

Para usar `requests` necesitas internet y tenerla instalada (`pip install requests`). El interactivo de esta semana maneja el caso de "no hay internet" sin romperse.

11.9. Resumen de la semana

```
# Importar módulos estándar
import math, random, json, os
from datetime import date

math.sqrt(16)           # 4.0
random.choice(["a", "b"]) # uno al azar
json.dumps({"x": 1})     # '{"x": 1}'
date.today().isoformat() # fecha de hoy

# Módulo propio
import pokeutils
pokeutils.formatear_nombre("pikachu")

# Instalar librerías
# pip install requests
# pip install -r requirements.txt

# Consumir una API
import requests
datos = requests.get("https://pokeapi.co/api/v2/pokemon/pikachu").json()
```

Concepto**Para qué sirve**`import`

Traer código de otros

Concepto	Para qué sirve
<code>math/random/json/os/datetime</code>	Módulos estándar
módulo propio	Tu propio <code>.py</code> reutilizable
<code>pip</code>	Instalar librerías externas
<code>venv</code>	Entorno aislado por proyecto
<code>requirements.txt</code>	Lista de dependencias
<code>requests</code>	Pedir datos a una API

11.10. ¿Y ahora qué?

1. Resolvé `ejercicios.py`.
2. Mirá el módulo de ejemplo `pokeutils.py`.
3. Corré los tests: `pytest semana-10-python-modulos-y-pip/`
4. Instalá requests (`pip install requests` o `bash ../setup.sh`) y jugá la **Pokédex online**:

```
python interactivo.py
```

ÁNIMO

"El mejor programador no es el que escribe más código, sino el que sabe reusar el de los demás."

CAPÍTULO 12. Mapa de temas del curso

Un vistazo a **todo el recorrido**, semana por semana. Cada semana tiene su teoría (en este libro), ejercicios, soluciones, tests y un programa interactivo.

12.1. Fase 1 — Linux

Semana	Temas
01 — Fundamentos	Terminal, navegación, <code>ls cd pwd mkdir rm cp mv cat echo</code> , rutas, permisos
02 — Intermedio	<code>nano</code> , variables, scripts bash, <code>> >> </code> , <code>grep find ps kill chmod apt</code> , SSH

12.2. Fase 2 — Python básico

Semana	Temas
03 — Introducción	Variables, tipos, <code>print</code> , <code>input</code> , f-strings
04 — Control de flujo	<code>if/elif/else</code> , comparadores, lógicos, <code>while/for/range</code> , <code>break/continue</code>
05 — Funciones	<code>def</code> , parámetros, <code>return</code> , scope, <code>lambda</code> , recursión
06 — Listas y colecciones	Listas, tuplas, sets, diccionarios, comprensiones, <code>enumerate</code> , <code>zip</code>
07 — Cadenas y archivos	Métodos de string, slicing, <code>open</code> , CSV, <code>try/except</code>

12.3. Descanso — Git

Git: Control de Versiones — `init`, `add`, `commit`, `log`, ramas (`branch/switch/merge`), GitHub, `push/pull/clone`, `.gitignore`. Pensada para hacerse después de la semana 7.

12.4. Fase 3 — Python avanzado

Semana	Temas
08 — POO introducción	Clases, <code>__init__</code> , atributos, métodos, <code>self</code> , <code>__str__</code> , <code>__repr__</code>
09 — POO avanzado	Herencia, <code>super()</code> , polimorfismo, <code>@property</code> , clases abstractas (<code>abc</code>)
10 — Módulos y pip	<code>import</code> , módulos estándar, módulos propios, <code>pip</code> , <code>venv</code> , <code>requests</code>

12.5. Fase 4 — Proyectos

- **Semana 11 — Agenda del Entrenador:** app de consola modular con persistencia en JSON.
- **Semana 12 — Pokédex Web:** web con Flask + PokéAPI.
- **proyectos/pokedex-cli:** Pokédex de consola con sprites ASCII y favoritos.
- **proyectos/batalla-pokemon:** simulador con tipos, PP y estados alterados.
- **proyectos/agenda-entrenador** y **proyectos/pokedex-web:** versiones pulidas.

CAPÍTULO 13. Ayuda: tests, preguntas frecuentes y glosario

13.1. Cómo correr los tests

Los **tests** te dicen si tus ejercicios están bien. Usamos **pytest**:

```
pytest # todos los tests
pytest curso/semana-04-python-control-de-flujo/ # una semana
pytest curso/semana-05-python-funciones/test_ejercicios.py -v # con detalle
```

NOTA

Verde = aprobado. Rojo = revisá tu solución. Por defecto los tests prueban las soluciones; la Liga los corre contra TU `ejercicios.py` para darte EXP y decirte qué ejercicio falta.

13.2. Preguntas frecuentes

¿Por dónde empiezo? Por la Liga: `python aventura.py`. Después seguí la semana 01.

Me sale "command not found: python". Probá con `python3`.

Los tests me dan rojo. ¿Está mal? Significa que tu solución todavía no es correcta. Leé el error (está en español), corregí y volvé a probar. Es parte normal del aprendizaje.

¿Puedo ver las soluciones? Sí, están en `soluciones.py`. Pero intentá primero vos: mirar sin intentar es como usar un truco, ganás pero no aprendés.

¿Necesito internet? Solo para algunas partes (la semana 10, la Pokédex online y el autocompletado web). El resto funciona sin conexión.

13.3. Glosario

Palabra	Qué significa
Terminal / consola	La ventana donde escribís comandos de texto.
Variable	Una "caja con nombre" donde guardás un dato.

Palabra	Qué significa
Tipo	La clase de dato: int, float, str, bool...
Función	Un bloque de código con nombre que podés reusar.
Parámetro / argumento	El dato que recibe / le pasás a una función.
Bucle	Una estructura que repite código (while, for).
Lista / diccionario	Colecciones que guardan muchos datos.
Clase / objeto	El molde / un individuo hecho con ese molde (POO).
Test	Un programa que revisa si tu código funciona bien.
EXP	Puntos de experiencia que ganás en la Liga al pasar tests.
Commit	Un punto de guardado de tu código en Git.
venv	Entorno virtual: cajita aislada con las librerías del curso.

"El mejor momento para empezar a programar fue ayer. El segundo mejor momento es ahora."
¡A atraparlos a todos!